

ЛАБОРАТОРНА РОБОТА №1. ТИПИ ДАНИХ PYTHON

Мета: Закріпити навички використання базових та спеціалізованих типів даних Python, навчитися обирати оптимальні структури для вирішення інженерних завдань та аналізувати їхню продуктивність.

ЧАСТИНА 1 (базова): Базові операції (оцінюється у 4 бали)

Завдання 1.1 "Очищувач тексту" (String & List & Set): На базі заданого тексту (файл bath-livingston.txt) отримати очищений список (list) усіх слів (зберігаючи дублікати та порядок). Відфільтрувати дані: видалити всі знаки пунктуації (якщо вони залишились) та всі артикли ("a" / "the"). Потім перетворити його на множину (set), щоб отримати унікальний словник.

Завдання 1.2 "Аналіз асортименту" (String & List & Set): Дано два списки артикулів товарів із двох складів (файл assortment-analysis.py). Знайдіть:

- Унікальні товари, що представлені лише на одному зі складів (симетрична різниця).
- Кількість унікальних товарів.
- Товари, які є хоча б на одному зі складів.
- Товари, які є на першому складі, але відсутні на другому.
- Товари, що є на обох складах: результат надати у вигляді списку.
- Відсортований (лексикографічно) перелік усіх товарів, при цьому артикул кожного товару повинен бути перетворений таким чином, щоб між буквенними символами та цифровими з'явився дефіс, наприклад, EL1005 -> EL-1005, DNF3761 -> DNF-3761.

Завдання 1.3 "Чистий список" (List): На вхід поступають дані про ціни з сайту-агрегатора (файл clean-list.py). Через особливості пагінації деякі товари потрапили у список двічі, наприклад: [['iPhone 15', 35000], ['Samsung S24', 32000], ['iPhone 15', 35000]].. Необхідно реалізувати очищення списку від дублікатів in-place (без створення копії списку).

Завдання 1.4 "Частотний аналіз" (Dict & Tuple): Використовуючи надані тексти (файли bester-eng.txt, bester-rus.txt), реалізуйте розширений аналіз вживаності мовних одиниць.

Аналіз слів (Уніграми):

- Підрахувати кількість входжень кожного слова окремо для текстів різними мовами, попередньо відфільтрувавши текст від знаків пунктуації, чисел та спецсимволів.
- Знайти Топ-3 найпопулярніших слова.

Контекстний аналіз (Біграми):

- Сформуувати частотний словник біграм (пар слів, що йдуть підряд).
- Технічна умова: Ключем словника має бути кортеж (word1, word2).
- Вивести 5 найчастіших словосполучень (біграм) для кожної мови.

Символьний аналіз:

- Побудувати ранжований перелік символів за частотою їх появи.

Практичні кейси:

- *Інтелектуальне введення тексту (T9/Autofill)*: Саме на основі біграм та триграм ваш смартфон пропонує наступне слово під час друку. Якщо пара "Добрий" + "день" зустрічається часто, система запропонує "день" одразу після введення першого слова.
- *Виявлення плагіату та стилometрія*: У кожного автора є унікальні "цифрові відбитки" — специфічні пари слів, які він використовує найчастіше. Аналіз біграм дозволяє з високою ймовірністю встановити авторство анонімного тексту.
- *Машинний переклад*: Сучасні системи перекладу аналізують контекст (сусідні слова), щоб обрати правильне значення багатозначного слова. Наприклад, біграма "bank account" та "river bank" допоможуть системі зрозуміти, чи йдеться про фінанси, чи про географію.
- *Стиснення даних*: Алгоритми можуть замінювати часті пари слів короткими кодами, що значно зменшує розмір текстових файлів.

ЧАСТИНА 2 (середній рівень): оцінюється у 3 бали.

Завдання 2.1 "Дворівневий реєстр": Ви працюєте над модулем електронного журналу. Система отримує результати іспитів у вигляді довгого «плаского» списку записів: ('Shevchenko', 'Data Bases', 95), ('Petrenko', 'Operating Systems', 88), Для генерації звітів та інтерфейсу викладача потрібно швидко отримувати оцінку студента за конкретну дисципліну.

Ваша задача: Розробити алгоритм трансформації цього списку у вкладену структуру даних, яка забезпечує миттєвий доступ до оцінки за ключами.

Вимоги:

- Отримання оцінки має виконуватися за складністю $O(1)$ відносно кількості студентів та дисциплін.
- Вихідні дані знаходяться у файлі two-level-register.py.

Завдання 2.2 "Role-Based Access Control (RBAC)": Ви розробляєте модуль безпеки для корпоративного порталу. Користувач може одночасно мати кілька ролей (наприклад, ["developer", "manager"]). Кожна роль має свій набір атомарних

дозволів (наприклад, read, write, execute).

Ваша задача: Реалізувати систему, яка обчислює підсумковий набір прав для користувача (об'єднання всіх прав його ролей) та забезпечує миттєвий доступ ($O(1)$) при повторних запитах з тим самим набором ролей.

Вимоги:

- Підсумкові права — це **об'єднання** (union) множин дозволів усіх ролей користувача.
- Порядок ролей у запиті **не має значення** (Запит для ["admin", "editor"] та ["editor", "admin"] має повертати той самий об'єкт із кешу).
- Перший виклик для нового набору ролей може бути повільним (обчислення), але всі наступні мають бути $O(1)$ (**отримання з кешу**).
- Для реалізації кешу використовувати словник (dict), ключами в якому будуть набори ролей.
- Вихідні дані для з переліком ролей та правами для кожної з них знаходяться у файлі rbac.py.
- При тестуванні дотримуватись наступного плану:
 - 1) Функція приймає список ролей.
 - 2) Перевіряє наявність у словнику-кеші.
 - 3) Якщо там порожньо, то додає відповідний ключ до словнику.
- Перелік запитів для тестування брати в тому ж файлі rbac.py.

Завдання 2.3 “Дешифратор Віженера”: Ви перехопили повідомлення, зашифроване за допомогою класичного шифру Віженера. Попередньо текст було приведено до нижнього регістру та видалено всі пробіли і знаки пунктуації. Вам відомий ключ шифрування — python.

Ваша задача: відновити текст.

Теоретичні відомості: Шифр Віженера використовує циклічне накладання ключа на текст. Кожна літера ключа визначає величину зсуву алфавіту (де $a=0$, $b=1$, $p=15$ і т.д.).

Приклад:

Текст: helloworld

Ключ: abr (повторюється як абрабравбра)

Крок	Текст	Ключ	Зсув	Розрахунок	Результат
1	h	a	0	'h' + 0	h
2	e	b	1	'e' + 1	f
3	l	p	15	'l' + 15	a (цикл через Z)
4	l	a	0	'l' + 0	l
5	o	b	1	'o' + 1	p
6	w	p	15	'w' + 15	l (цикл через Z)
...	...	...	...	...	...

Результат: hfalplosad

Технічні вимоги:

- Алфавіт: Використовуйте `string.ascii_lowercase` для роботи з символами.
- Ефективність: Для маніпуляцій з алфавітом використовуйте структуру `collections.deque`. Замість перерахунку індексів, реалізуйте зсув через метод `.rotate()`.
- Порівняння: У звіті обґрунтуйте, чому використання `rotate()` для черги є більш ефективним за створення нових зрізів (slicing) рядка на кожній ітерації.
- Вхідні дані: Зашифрований текст знаходиться у файлі `vigenere.txt`.

Завдання 2.4 “Бінарне шифрування XOR та робота з типом bytes”

Контекст: Операція XOR (виключне АБО) є базовим інструментом криптографії завдяки властивості інволюції: якщо $A \oplus B = C$, то $C \oplus B = A$. Це дозволяє використовувати один і той самий код як для зашифрування, так і для дешифрування даних.

Ваша задача:

- Перевірити роботу алгоритму шляхом шифрування та подальшого дешифрування будь-якого текстового рядка та будь-якого ключа. Якщо все зроблено правильно, результат дешифрування повинен співпасти з початковим рядком.
- Розшифрувати повідомлення, яке міститься у файлі `xor-message.bin` за допомогою ключа “python” (повідомлення представлено в файлі в бінарному вигляді). Написати функцію, яка приймає вхідне повідомлення та ключ у вигляді об'єктів `bytes`.

Методичні вказівки:

- В мові Python операція виключного АБО (XOR) представлена оператором “^”, який застосовується до окремих цілих чисел (елементів послідовності `bytes`) і працює побітово: результат операції буде дорівнювати 1 тільки в тому випадку коли в точності один з операндів дорівнює 1, наприклад:

`10010111^01010011=11000100`

- Для коректної роботи оператора “^” як текст так і ключ шифрування повинні бути представлені послідовністю байтів (типи даних `bytes` або `bytearray`), наприклад,

```
msg = 'Hello, World!'
b_msg = msg.encode('utf-8') # або b_msg = b'Hello, World!' (тільки для латиниці)
key = 'custom key'
b_key = key.encode('utf-8') # або b_key = b'custom key' (тільки для латиниці)
```

- Ключ має накладатися **циклічно**.

- При роботі з бінарними файлами використовуйте бінарний режим читання/запису ('rb'/'wb').
- Для коректного відображення результат дешифрування потрібно конвертувати в utf-8: `b_mes.decode('utf-8')`.

Практичні сценарії використання XOR

- 1) *Маскування даних (Data Obfuscation)* – найпоширеніший сценарій у розробці ПЗ – XOR використовується не для захисту від хакерів, а для того, щоб зробити дані нечитабельними для очей або простих автоматичних сканерів.

Приклад: Зберігання рядків (паролів до БД, API-ключів або конфігурацій) всередині бінарного файлу програми. Якщо просто відкрити такий файл у текстовому редакторі, ви нічого не побачите, бо дані «замасковані» XOR-ключем.

Антивіруси: Шкідливе ПЗ часто шифрує свої модулі за допомогою XOR, щоб антивірусні сканери не могли знайти сигнатури (відомі фрагменти коду) у файлі.

- 2) *Протоколи зв'язку та IoT (Інтернет речей):* у маленьких пристроях (датчики температури, розумні лампочки), де процесор дуже слабкий, повноцінне шифрування (наприклад, AES) може сильно сповільнювати роботу або швидко розряджати акумулятор.

Сценарій: Швидке маскування пакетів даних, що передаються через Wi-Fi або Bluetooth, щоб сторонній спостерігач не міг миттєво перехопити сенсорні дані.

- 3) *RAID-масиви (Відновлення даних):* технологія RAID 5 використовує властивість XOR для відновлення даних у разі виходу з ладу одного з жорстких дисків.

Як це працює: Система обчислює результат XOR для блоків даних на різних дисках і записує його на «диск парності». Якщо один диск «помирає», дані на ньому можна математично відновити, виконавши XOR між дисками, що залишилися, та блоком парності.

- 4) *Графічні операції та ігри:* в комп'ютерній графіці XOR використовується для малювання об'єктів (наприклад, курсора миші або рамки виділення), які можна легко видалити з екрана, не перемальовуючи все зображення.

Механіка: Коли ви малюєте об'єкт методом XOR поверх фону, повторне накладання того самого об'єкта тим самим методом XOR повністю відновлює оригінальний фон. Це ідеально для динамічних елементів інтерфейсу.

- 5) *Одноразовий блокнот (One-Time Pad)*: це єдиний вид шифрування, який вважається **теоретично неможливим для злому**, якщо дотримано умови: ключ є абсолютно випадковим, дорівнює довжині повідомлення і використовується лише один раз.

Механіка: Побітовий XOR між повідомленням і цим випадковим ключем. Без ключа отримати вихідний текст математично неможливо, незалежно від потужності комп'ютера.

ЧАСТИНА 3 (високий рівень): оцінюється у 3 бали.

Завдання 3.1 Аналіз схожості покупців (Рекомендаційна система)

Контекст: Уявіть маркетплейс із мільйоном товарів. Кожен покупець взаємодіє лише з невеликою кількістю позицій (купує 5-10 товарів). Якби ми зберігали це у звичайній таблиці, вона б складалася на 99% з нулів. В інженерії це називається розрідженими даними. Ваша задача — знайти «схожих» покупців, щоб рекомендувати їм товари один одного.

Ваша задача:

- *Збереження даних:* Використати структуру `dict[user_id, dict[item_id, count]]`.
Наприклад: `{ 'user1': { 'apple': 5, 'banana': 2 }, 'user2': { 'apple': 3, 'orange': 1 } }`.
- *Розрахунок:* Реалізувати функцію, яка обчислює схожість між двома довільними `user_id`.
Алгоритм має бути оптимізованим: для чисельника ітеруйтеся тільки за списком товарів меншого профілю та перевіряйте наявність цього товару в іншого.
- *Видалення:* Реалізувати функцію `update_purchase(user_id, item_id, delta)`. Якщо після зміни `delta` кількість стає 0 або менше — товар має бути видалений зі словника користувача.
- *Аналіз продуктивності обраних структур даних:* Проаналізуйте час підрахунку схожості користувачів шляхом побудови таблиці (або графіку) залежності часу виконання від кількості даних (10^2 , 10^4 , 10^6 елементів). Для часових замірів використовуйте `time.perf_counter()`.

На що звернути увагу:

- *Тест на «порожній перетин»:* Як поводить ся програма, якщо у користувачів немає жодного спільного товару? (Результат має бути 0.0, а не помилка `DivisionByZero`).
- *Тест на ідентичність:* Схожість користувача з самим собою має бути рівною 1.0.
- *Ефективність циклу:* подумайте, по якому з 2-х профілів, для яких

обчислюється ефективність, робити ітерацію.

- *Аналіз часу доступу до елементів хеш таблиці:* Завдяки використанню хеш-таблиць (dict), час пошуку профілю користувача повинен бути сталим незалежно від кількості користувачів у базі.

Практичні кейси:

Amazon/Netflix: Рекомендації товарів або фільмів («Користувачі, схожі на вас, також купували...»).

Spotify: Складання персоналізованих плейлистів на основі схожості смаків.

Приклад вхідних/вихідних даних:

```
# Вхідні дані (профілі покупок):
storage = {
    "user_101": {"iPhone": 1, "AirPods": 1},
    "user_102": {"iPhone": 1, "MacBook": 1}
}

# Виклик функції:
result = get_similarity(storage, "user_101", "user_102")

# Розрахунок:
# Чисельник (спільний iPhone): 1 * 1 = 1
# Довжина user_101: sqrt(1^2 + 1^2) = 1.414
# Довжина user_102: sqrt(1^2 + 1^2) = 1.414
# Результат: 1 / (1.414 * 1.414) = 0.5
```

Методичні вказівки:

Математична модель (Косинусна схожість): Схожість двох користувачів А та В визначається як косинус кута між їхніми векторами інтересів. Чим ближче значення до 1, тим більше схожі їхні смаки. Формула схожості:

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

де $A \cdot B$ — скалярний добуток (сума добутків спільних значень): потрібно знайти товари, які купили обидва користувачі, перемножити їхні кількості (або рейтинги) для кожного такого товару та додати ці добутки., а $\|A\|$ та $\|B\|$ — евклідові довжини векторів (корінь із суми квадратів усіх значень вектора).

Приклад розрахунку схожості:

User_A = {"item1": 5, "item2": 3}

User_B = {"item1": 2, "item3": 10}

Чисельник: $5 \times 2 = 10$ (тільки item1 спільний).

Довжина A: $\sqrt{5^2 + 3^2} = \sqrt{34} \approx 5.83$.

Довжина B: $\sqrt{2^2 + 10^2} = \sqrt{104} \approx 10.19$.

Схожість: $10 / (5.83 \times 10.19) = 0.168$.

Скрипт для генерування тестових даних:

```
import random
```

```
def generate_task8_data(num_users=10000):
    """Генерація даних для рекомендаційної системи """
    categories = ["Laptop", "Smartphone", "Headphones", "Camera", "Monitor", "Watch"]
    brands = ["Apple", "Samsung", "Sony", "Dell", "HP", "Logitech", "LG", "Canon"]
    models = ["Pro", "Air", "Ultra", "Max", "Plus", "GaminX", "Elite", "Series 5"]
    # Формуємо каталог: 8 brands * 6 categories * 8 models * 2 variants = 768 товарів
    catalog = [f"{b} {c} {m} {i}" for b in brands
               for c in categories
               for m in models
               for i in range(2)]
    storage = {}
    for i in range(num_users):
        u_id = f"user_{i:05d}"
        # Вибираємо від 3 до 15 унікальних товарів для кожного користувача
        n_items = random.randint(3, 15)
        bought = random.sample(catalog, n_items)
        # Кожному товару присвоюємо випадкову кількість (від 1 до 5 одиниць)
        storage[u_id] = {item: random.randint(1, 5) for item in bought}
    return storage
```

Завдання 3.2 Повнотекстовий пошук (In-Memory Inverted Index)

Контекст: Під час аналізу мільйонів логів у системах типу Elasticsearch або Splunk пошук через простий перебір рядків працює занадто повільно. Для миттєвої відповіді використовується інвертований індекс — структура, що пов'язує кожне окреме слово з набором ідентифікаторів записів, де воно зустрічається.

Ваша задача: Створити систему індексації та швидкого пошуку за повідомленнями.

1) Токенізація та Індексація:

- Програма має опрацювати кожне повідомлення (message) у списку логів.
- Повідомлення потрібно розбити на окремі слова (токени), привести до нижнього регістру та видалити знаки пунктуації.
- Створити індекс у вигляді словника, де ключем є слово (рядок), а значенням — множина (set) ідентифікаторів (id) записів, у яких це слово зустрічається.

2) Пошуковий запит:

- Реалізувати функцію, яка приймає запит із кількох слів (наприклад, "failed" AND "auth-service").
- Пошук має виконуватися через перетин множин (intersection або оператор &) значень з індексу.

3) Вимірювання продуктивності:

- Порівняти час пошуку за конкретним унікальним TraceID двома

способами:

- a) Прямим перебором списку логів ($O(N)$): `log['trace_id'] == "TRC-012345"`.
- b) Через інвертований індекс ($O(1)$): `trace_index["TRC-012345"]`.
- Результати надати для обсягів 10^2 , 10^4 та 10^6 записів.
- Для часових замірів використовувати `time.perf_counter()`.

На що звернути увагу:

Якщо для час пошуку зростає лінійно в залежності від кількості логів, то з індекс побудовано невірно.

Приклад вхідних та вихідних даних

Вхідні дані (Список логів): Кожен лог — це словник. Зверніть увагу, що пошук може йти як за полем `trace_id` (точний пошук), так і за словами всередині `message` (повнотекстовий пошук).

```
logs = [  
    {  
        "id": 0,  
        "trace_id": "TRC-101",  
        "service": "auth-gateway",  
        "message": "User login failed: invalid password."  
    },  
    {  
        "id": 1,  
        "trace_id": "TRC-102",  
        "service": "payment-processor",  
        "message": "Payment failed: connection timeout."  
    },  
    {  
        "id": 2,  
        "trace_id": "TRC-103",  
        "service": "auth-gateway",  
        "message": "User login success."  
    }  
]
```

Вихідні дані (Побудований індекс): Після токенизації повідомлень (видалення знаків пунктуації, приведення до нижнього регістру) індекс слів у полі `message` матиме такий вигляд:

Ключ: токен (слово), Значення: множина (set) ID записів

```
index_message = {  
    "user": {0, 2},  
    "login": {0, 2},  
    "failed": {0, 1},  
    "invalid": {0},  
    "password": {0},  
    "payment": {1},  
    "connection": {1},  
    "timeout": {1},  
    "success": {2}
```

}

Результат пошукового запиту:

Якщо ми шукаємо "failed" AND "login":

index_message["failed"] → {0, 1}

index_message["login"] → {0, 2}

Результат ({0, 1} & {0, 2}): ID: 0 (Повертаємо лог із цим ID).

Варіативна складова завдання (реалізовується ЗА БАЖАННЯМ):

Реалізація «Розумного пошуку» (Smart Query Engine)

Контекст: В реальних системах аналітики (як-от Kibana або Grafana) користувач рідко шукає за одним словом. Зазвичай це комбінація фільтрів: «показати всі помилки сервісу Billing за конкретний сеанс».

Технічне завдання: Реалізувати функцію search_logs(storage, **filters), яка приймає довільну кількість іменованих аргументів.

Логіка роботи:

- Функція отримує набори ID записів для кожного фільтра з відповідних індексів.
- Якщо передано кілька фільтрів (наприклад, level="ERROR" та service="auth"), функція має повернути лише ті записи, що задовольняють всім умовам одночасно.
- Ключова вимога: Використовувати операцію перетину множин (set.intersection або оператор &). Це дозволяє миттєво відфільтрувати мільйони записів, не переглядаючи кожен із них у циклі.

Приклад використання:

Пошук за кількома критеріями одночасно

```
results = search_logs(  
    level="CRITICAL",  
    service="payment-processor",  
    trace_id="TRC-001234"  
)
```

Очікувана внутрішня логіка:

```
# ids = index_level["CRITICAL"] & index_service["payment-processor"] &  
#       index_trace["TRC-001234"]  
# return [all_logs[i] for i in ids]
```

Практичні кейси:

- Elasticsearch/Solr: Системи пошуку по великих масивах тексту.
- GitHub: Пошук за фрагментами коду серед мільйонів репозиторіїв.
- Бази даних: Прискорення запитів WHERE column LIKE '%word%'.

- *Основне завдання (Повнотекстовий пошук)*: Це про те, як працює пошук у Google. Ми беремо «сире» речення, очищуємо його від сміття (ком, крапок), розбираємо на окремі слова і миттєво знаходимо сторінки, де ці слова зустрічаються. Тут головний виклик — правильно «порізати» текст на шматочки (токенізація).
- *Варіативна складова (Smart Search)*: Це про те, як працюють фільтри в інтернет-магазині. Коли ви одночасно вибираєте «Бренд» та «Цінову категорію», система бере готові списки товарів для кожного фільтра і знаходить їхній спільний перетин. Тут головний виклик — об'єднати дані з декількох різних індексів одночасно.

Скрипт для генерування тестових даних:

```
import datetime
import random

def generate_task9_data(num_logs=1000000):
    levels = ["INFO", "DEBUG", "WARNING", "ERROR", "CRITICAL"]
    services = ["auth-gateway", "payment-processor", "inventory-db",
"user-session-manager"]
    actions = ["connected to", "failed to process", "successfully updated", "timed
out during", "rejected request from"]
    resources = ["database", "s3-bucket", "external-api", "redis-cache",
"oauth-provider"]
    trace_ids = [f"TRC-{i:06d}" for i in range(50000)]
    logs = []
    # Початковий час для імітації хронології
    start_time = datetime.datetime.now()
    for i in range(num_logs):
        # Структурований об'єкт логу
        log_entry = {
            "id": i,
            "timestamp": (start_time +
datetime.timedelta(milliseconds=i*10)).isoformat(),
            "level": random.choice(levels),
            "service": random.choice(services),
            "trace_id": random.choice(trace_ids),
            "message": f"Service {random.choice(services)} {random.choice(actions)}
{random.choice(resources)}"
        }
        logs.append(log_entry)
    return logs
```

Завдання 3.3 Синхронізація станів: Рекурсивний Diff вкладених структур

Контекст: У розподілених системах (по типу Cassandra) та інструментах спільної роботи (Git, Google Docs) важливо передавати не весь файл, а тільки дельту (зміни). Також потрібно вміти виявляти конфлікти, якщо двоє розробників змінили одне й те саме поле одночасно.

Ваша задача: Написати алгоритм порівняння двох версій складних налаштувань системи (вкладені словники).

- 1) Функція `get_diff(old_dict, new_dict)`: Рекурсивно обходить словники та повертає словник змін.
- 2) Формат дельти: Результат має містити шлях до поля (наприклад, `settings.theme`) та його нове значення.
- 3) Merge & Conflict:
 - Дано три об'єкти: Базова версія (V_0), зміни Користувача А (V_1) та зміни Користувача Б (V_2).
 - Потрібно автоматично об'єднати зміни V_1 та V_2 у фінальну версію.
 - Якщо обидва користувачі змінили одне й те саме поле на різні значення — функція має повернути список конфліктів.
- 4) Продемонструйте роботу алгоритму на двох наборах даних:
 - а) `V0 = {`

```
"system": {
  "network": {"ip": "192.168.1.1", "dns": "8.8.8.8"},
  "display": {"brightness": 70, "theme": "light"}
},
"user": {"name": "Admin", "role": "root"}
}
```

`V1, V2` сформувати самостійно на базі `V0`, передбачивши випадки з конфліктами та без
 - б) Згенерований наданим в кінці завдання скриптом словник глибиною 5 рівнів. Програма має знайти зміни навіть у найглибших вузлах і коректно вивести шлях у форматі `node_0.node_1.node_3`.

Примітка щодо структури даних: У реальних інженерних системах (конфігурації хмарних сервісів, JSON-відповіді складних API) дані рідко мають «пласку» структуру. Зазвичай це дерево з багатьма рівнями вкладеності.

Ваше рішення має працювати з довільною глибиною вкладеності. Це перевіряє коректність вашої рекурсивної логіки: алгоритм повинен однаково успішно знаходити зміни як на першому рівні, так і на п'ятому чи десятому. Обробка глибоких структур дозволяє мінімізувати обсяг даних, що передаються мережею, замінюючи передачу всього об'єкта передачею точкової «дельти».

Приклад тестування:

- `V0: {"display": {"brightness": 70, "color": "warm"}}`
- User A змінив `brightness` на 80.
- User B змінив `color` на `cold`.
- Результат: `{"display": {"brightness": 80, "color": "cold"}}`.

- Зміни відбулися в одній секції (display), але в різних полях, тому конфлікту немає.

Практичні кейси:

- Git: Порівняння версій коду та об'єднання гілок (merge).
- Google Docs: Одночасне редагування документа кількома користувачами.
- Мобільні додатки: Синхронізація налаштувань профілю між телефоном та веб-версією.

Скрипт для генерування тестових даних:

```
import random
import copy

# 1. Скрипт для "технічного стрес-тесту" (будь-яка глибина)
def get_random_path(d, current_path=None):
    """Знаходить випадковий шлях до листа в словнику"""
    if current_path is None:
        current_path = []
    if not isinstance(d, dict):
        return current_path
    key = random.choice(list(d.keys()))
    current_path.append(key)
    return get_random_path(d[key], current_path)

def set_by_path(d, path, value):
    """Встановлює значення за вказаним шляхом"""
    for key in path[:-1]:
        d = d[key]
    d[path[-1]] = value

def generate_task3_stress_test(depth=5, width=3):
    # Генеруємо базу
    def gen(d):
        if d == 0: return random.randint(1, 100)
        return {f"node_{i}": gen(d-1) for i in range(width)}

    v0 = gen(depth)
    v1, v2 = copy.deepcopy(v0), copy.deepcopy(v0)

    # Автоматично змінюємо 3 випадкові гілки для Користувача А
    for _ in range(3):
        path = get_random_path(v0)
        set_by_path(v1, path, random.randint(1000, 2000))

    # Автоматично змінюємо 3 випадкові гілки для Користувача Б
    for _ in range(3):
        path = get_random_path(v0)
        set_by_path(v2, path, random.randint(3000, 4000))

    return v0, v1, v2
```

КОНТРОЛЬНІ ЗАПИТАННЯ

1. Поясніть різницю між мутабельними (mutable) та імутабельними (immutable) типами даних у Python. Наведіть приклади.
2. У чому головна відмінність між list (список), set (множина) та tuple (кортеж) з точки зору впорядкованості та можливості зміни елементів?
3. Яку структуру даних ви оберете, якщо потрібно гарантувати відсутність дублікатів у переліку артикулів товарів? Чому?
4. Які типи даних можуть бути ключами словника, а які — ні? Чому не можна використовувати список як ключ?
5. Чому пошук елемента в множині (set) або за ключем у словнику (dict) працює швидше, ніж пошук у списку (list) серед мільйона записів?
6. Як зміниться швидкість пошуку оцінки студента, якщо замість вкладеного словника (dict) ми будемо шукати її простим перебором у списку кортежів?
7. У чому полягає різниця між об'єктом типу str та об'єктом типу bytes? І яку роль відіграють оператори .encode() та .decode()?
8. У чому перевага виконання операцій in-place (як у завданні з очищенням списку цін)? У яких сценаріях варто уникати створення копій великих об'єктів і як подібна вимога вплине на вибір типів даних?